Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 12: Namespaces, Scope & Decorators

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Namespaces
2. Scope & LEGB Rule
3. Local & Global Scope
4. Built-in Scope
5. Enclosing Scope & nonlocal
6. Decorators Basics
7. Practical Decorators (Timer, Login, Validation)
8. Multiple & Class Decorators

48 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 12: Namespaces, Scope & Decorators - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Namespaces, Scope & LEGB Rule, Local & Global Scope, Built-in Scope, Enclosing Scope & nonlocal, Decorators Basics, Practical Decorators (Timer, Login, Validation), Multiple & Class Decorators
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Namespaces

Q1. Global Namespace

BEGINNER

Create two global variables shop = "Bikkad Cafe" and city = "Pune". Check whether they appear in globals().

Solution:

```
shop = "Bikkad Cafe"
city = "Pune"

print("shop" in globals())
print(globals()["city"])
```

Output:

```
True
Pune
```

Q2. Local Namespace

BEGINNER

Inside a function make_bill(), create item and price. Print locals() inside the function.

Solution:

```
def make_bill():
    item = "Tea"
    price = 15
    print(locals())

make_bill()
```

Output:

```
{'item': 'Tea', 'price': 15}
```

Q3. Built-in Namespace

INTERMEDIATE

Check whether "print", "len" and "shop" are present in the built-in namespace using dir(builtins).

Solution:

```
import builtins

names = dir(builtins)
print("print" in names)
print("len" in names)
print("shop" in names)
```

Output:

```
True
True
False
```

Q4. Same Name in Different Namespaces

INTERMEDIATE

Create a global total = 100 and a local total = 5 inside a function. Print both and explain using comments.

Solution:

```
total = 100             # global namespace

def calc():
    total = 5           # local namespace of calc
    print(total)

calc()
print(total)
```

Output:

```
5
100
```

Q5. Each Call Gets a New Namespace

ADVANCED

Write counter() that creates count = 0 and adds 1. Call it 3 times and show it always prints 1 (a fresh local namespace every call).

Solution:

```
def counter():
    count = 0
    count += 1
    print(count)

counter()
counter()
counter()
```

Output:

```
1
1
1
```

Q6. Module Namespace

ADVANCED

Import math and show that pi lives in the math module's namespace: compare math.pi with vars(math)["pi"] and check "pi" in globals().

Solution:

```
import math

print(math.pi == vars(math)["pi"])
print("pi" in globals())
print("math" in globals())
```

Output:

```
True
False
True
```

2. Scope & LEGB Rule

Q7. Local Wins

BEGINNER

Create a global x = "global" and a function with local x = "local". Print x inside the function.

Solution:

```
x = "global"

def show():
    x = "local"
    print(x)

show()
```

Output:

```
local
```

Q8. Global Used When No Local

BEGINNER

Create global tax = 18 and a function that prints tax without defining it locally.

Solution:

```
tax = 18

def show_tax():
    print(tax)

show_tax()
```

Output:

```
18
```

Q9. Full LEGB Chain

INTERMEDIATE

Create a variable name at global level, at enclosing level (outer) and at local level (inner). Print from inner, then remove the local and see which one is used.

Solution:

```
name = "Global"

def outer():
    name = "Enclosing"

    def inner():
        name = "Local"
        print(name)

    def inner2():
        print(name)

    inner()
    inner2()

outer()
```

Output:

```
Local
Enclosing
```

Q10. Built-in is the Last Level

INTERMEDIATE

Use len() inside a nested function where no level defines len. Python finds it in the Built-in scope.

Solution:

```
def outer():
    def inner():
        return len("Bikkad")
    return inner()

print(outer())
```

Output:

```
6
```

Q11. NameError When Not Found

ADVANCED

Print an undefined variable inside a nested function and catch the NameError, showing LEGB search failed.

Solution:

```
def outer():
    def inner():
        return discount
    return inner()

try:
    outer()
except NameError as e:
    print(e)
```

Output:

```
name 'discount' is not defined
```

Q12. Trace the LEGB Lookup

ADVANCED

Write a function find(name) that checks a local dict, an enclosing dict, globals() and builtins in order and prints where the name was found. Test with "price", "tax", "print".

Solution:

```
import builtins

tax = 18

def outer():
    enclosing = {"price": 500}

    def find(name):
        local = {"qty": 2}
        if name in local:
            return "Local"
        if name in enclosing:
            return "Enclosing"
        if name in globals():
            return "Global"
        if hasattr(builtins, name):
            return "Built-in"
        return "Not found"

    for n in ["price", "tax", "print", "xyz"]:
        print(n, find(n))

outer()
```

Output:

```
price Enclosing
tax Global
print Built-in
xyz Not found
```

3. Local & Global Scope

Q13. Local Variable Disappears

BEGINNER

Create a local variable otp inside generate() and try to print it outside inside try-except.

Solution:

```
def generate():
    otp = 4821
    print(otp)

generate()
try:
    print(otp)
except NameError as e:
    print(e)
```

Output:

```
4821
name 'otp' is not defined
```

Q14. Read a Global Inside a Function

BEGINNER

A global company = "Bikkad IT" is printed inside a function header().

Solution:

```
company = "Bikkad IT"

def header():
    print("Welcome to", company)

header()
```

Output:

```
Welcome to Bikkad IT
```

Q15. Modify a Global with global

INTERMEDIATE

A global stock = 50. Write sell(qty) that reduces it using the global keyword. Sell 12 and 8.

Solution:

```
stock = 50

def sell(qty):
    global stock
    stock -= qty

sell(12)
sell(8)
print(stock)
```

Output:

```
30
```

Q16. Create a Global from a Function

INTERMEDIATE

Inside setup() use global to create a new global variable app_version = "1.0". Print it outside after calling setup().

Solution:

```
def setup():
    global app_version
    app_version = "1.0"

setup()
print(app_version)
```

Output:

```
1.0
```

Q17. Global Mutable Without global

ADVANCED

A global list cart = []. Write add(item) that appends without using the global keyword. Explain why it works.

Solution:

```
cart = []

def add(item):
    cart.append(item)   # changes the object, does not rebind name

add("Pen")
add("Book")
print(cart)
```

Output:

```
['Pen', 'Book']
```

Q18. Avoid Globals: Pass and Return

ADVANCED

Rewrite the stock example without global: sell(stock, qty) returns the new stock.

Solution:

```
def sell(stock, qty):
    return stock - qty

stock = 50
stock = sell(stock, 12)
stock = sell(stock, 8)
print(stock)
```

Output:

```
30
```

4. Built-in Scope

Q19. Built-in Functions Everywhere

BEGINNER

Use max(), min() and sum() inside a function without importing anything.

Solution:

```
def report(marks):
    print(max(marks))
    print(min(marks))
    print(sum(marks))

report([67, 89, 72])
```

Output:

```
89
67
228
```

Q20. List Some Built-ins

BEGINNER

Print how many names exist in the builtins module and check that "input" is one of them.

Solution:

```
import builtins

names = dir(builtins)
print(len(names) > 100)
print("input" in names)
```

Output:

```
True
True
```

Q21. Shadowing a Built-in

INTERMEDIATE

Assign sum = 10 by mistake and then try to use sum([1, 2]) inside try-except. Fix it with del sum.

Solution:

```
sum = 10
try:
    print(sum([1, 2]))
except TypeError as e:
    print(e)

del sum
print(sum([1, 2]))
```

Output:

```
'int' object is not callable
3
```

Q22. Shadowing Only Inside a Function

INTERMEDIATE

Inside a function, create a local variable named len = 5. Show that outside the function len() still works.

Solution:

```
def test():
    len = 5
    print(len)

test()
print(len("Pune"))
```

Output:

```
5
4
```

Q23. Access the Original Built-in

ADVANCED

After shadowing max with a variable, call the real max using builtins.max.

Solution:

```
import builtins

max = "not a function"
print(builtins.max(4, 9, 2))
del max
```

Output:

```
9
```

Q24. Find Shadowed Names

ADVANCED

Given a list of variable names a student used ["list", "total", "str", "price", "id"], print the ones that shadow built-ins.

Solution:

```
import builtins

used = ["list", "total", "str", "price", "id"]
for name in used:
    if hasattr(builtins, name):
        print(name, "shadows a built-in")
```

Output:

```
list shadows a built-in
str shadows a built-in
id shadows a built-in
```

5. Enclosing Scope & nonlocal

Q25. Inner Reads Outer

BEGINNER

outer() has a variable msg = "Hello". inner() prints msg.

Solution:

```
def outer():
    msg = "Hello"

    def inner():
        print(msg)

    inner()

outer()
```

Output:

```
Hello
```

Q26. Inner Creates Its Own Variable

BEGINNER

Inside inner(), assign msg = "Hi" without nonlocal. Print msg in both functions.

Solution:

```
def outer():
    msg = "Hello"

    def inner():
        msg = "Hi"
        print(msg)

    inner()
    print(msg)

outer()
```

Output:

```
Hi
Hello
```

Q27. Change Outer Variable with nonlocal

INTERMEDIATE

Use nonlocal so inner() can increase the outer variable score by 10.

Solution:

```
def game():
    score = 0

    def hit():
        nonlocal score
        score += 10

    hit()
    hit()
    print(score)

game()
```

Output:

```
20
```

Q28. Closure Remembers Values

INTERMEDIATE

Write make_greeter(greeting) that returns a function. The returned function remembers greeting even after make_greeter finishes.

Solution:

```
def make_greeter(greeting):
    def greet(name):
        return greeting + ", " + name
    return greet

hello = make_greeter("Hello")
namaste = make_greeter("Namaste")
print(hello("Om"))
print(namaste("Pia"))
```

Output:

```
Hello, Om
Namaste, Pia
```

Q29. Counter Closure

ADVANCED

Write make_counter() that returns a function; each call returns the next number (1, 2, 3...). Create two independent counters.

Solution:

```python
def make_counter():
    count = 0

    def next_number():
        nonlocal count
        count += 1
        return count

    return next_number

c1 = make_counter()
c2 = make_counter()
print(c1(), c1(), c1())
print(c2())
```

Output:

```
1 2 3
1
```

Q30. Running Average Closure

ADVANCED

Write make_averager() returning a function add(value) that keeps all values and returns the current average. Add 80, 90, 70.

Solution:

```
def make_averager():
    values = []

    def add(value):
        values.append(value)
        return sum(values) / len(values)

    return add

avg = make_averager()
print(avg(80))
print(avg(90))
print(avg(70))
```

Output:

```
80.0
85.0
80.0
```

6. Decorators Basics

Q31. First Decorator

BEGINNER

Write a decorator stars that prints "*****" before and after the decorated function runs. Apply it to welcome().

Solution:

```
def stars(func):
    def wrapper():
        print("*****")
        func()
        print("*****")
    return wrapper

@stars
def welcome():
    print("Welcome to Bikkad IT")

welcome()
```

Output:

```
*****
Welcome to Bikkad IT
*****
```

Q32. Decorator Without @

BEGINNER

Apply the same stars decorator manually using welcome = stars(welcome).

Solution:

```
def stars(func):
    def wrapper():
        print("*****")
        func()
        print("*****")
    return wrapper

def welcome():
    print("Hello students")

welcome = stars(welcome)
welcome()
```

Output:

```
*****
Hello students
*****
```

Q33. Decorator for Functions with Arguments

INTERMEDIATE

Write a decorator log_call that prints the function name and arguments using *args, **kwargs, then calls the function.

Solution:

```
def log_call(func):
    def wrapper(*args, **kwargs):
        print("Calling", func.__name__, args, kwargs)
        return func(*args, **kwargs)
    return wrapper

@log_call
def add(a, b):
    return a + b

print(add(3, b=4))
```

Output:

```
Calling add (3,) {'b': 4}
7
```

Q34. Decorator that Changes the Result

INTERMEDIATE

Write a decorator uppercase that converts the returned string to uppercase. Apply to greet(name).

Solution:

```
def uppercase(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return result.upper()
    return wrapper

@uppercase
def greet(name):
    return "good morning " + name

print(greet("kavya"))
```

Output:

```
GOOD MORNING KAVYA
```

Q35. Keep Function Name with wraps

ADVANCED

Without functools.wraps the decorated function's __name__ becomes "wrapper". Show this and fix it using @wraps.

Solution:

```
from functools import wraps

def plain(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

def fixed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@plain
def report():
    pass

@fixed
def summary():
    pass

print(report.__name__)
print(summary.__name__)
```

Output:

```
wrapper
summary
```

Q36. Count Function Calls

ADVANCED

Write a decorator count_calls that counts how many times a function is called, stored as wrapper.calls.

Solution:

```
from functools import wraps

def count_calls(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@count_calls
def send_sms(number):
    return "Sent to " + number

send_sms("98111")
send_sms("98222")
send_sms("98333")
print(send_sms.calls)
```

Output:

```
3
```

7. Practical Decorators (Timer, Login, Validation)

Q37. Timer Decorator

BEGINNER

Write a decorator timer that measures how long a function takes using time.time(). Apply it to a function that sums 1 to 10 lakh.

Solution:

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print("Time taken:", round(end - start, 4), "seconds")
        return result
    return wrapper

@timer
def big_sum():
    return sum(range(1, 1000001))

print(big_sum())
```

Sample Output (changes on every run):

```
Time taken: 0.0127 seconds
500000500000
```

Q38. Login Required

BEGINNER

A global logged_in = False. Write a decorator login_required that runs the function only if logged_in is True, otherwise prints "Please login first".

Solution:

```
logged_in = False

def login_required(func):
    def wrapper(*args, **kwargs):
        if not logged_in:
            print("Please login first")
            return None
        return func(*args, **kwargs)
    return wrapper

@login_required
def view_profile():
    print("Showing profile")

view_profile()
logged_in = True
view_profile()
```

Output:

```
Please login first
Showing profile
```

Q39. Validate Positive Numbers

INTERMEDIATE

Write a decorator positive_only that raises ValueError if any argument is negative. Apply to area(l, w).

Solution:

```
def positive_only(func):
    def wrapper(*args):
        for a in args:
            if a < 0:
                raise ValueError("Negative values not allowed")
        return func(*args)
    return wrapper

@positive_only
def area(l, w):
    return l * w

print(area(5, 4))
try:
    area(5, -4)
except ValueError as e:
    print(e)
```

Output:

```
20
Negative values not allowed
```

Q40. Role-Based Access

INTERMEDIATE

Write a decorator admin_only(func) that checks user["role"] == "admin" (user is the first argument) before deleting a record.

Solution:

```
def admin_only(func):
    def wrapper(user, *args):
        if user["role"] != "admin":
            return "Access denied for " + user["name"]
        return func(user, *args)
    return wrapper

@admin_only
def delete_record(user, record_id):
    return f"Record {record_id} deleted by {user['name']}"

print(delete_record({"name": "Om", "role": "admin"}, 42))
print(delete_record({"name": "Pia", "role": "staff"}, 42))
```

Output:

```
Record 42 deleted by Om
Access denied for Pia
```

Q41. Decorator with Arguments (repeat)

ADVANCED

Write repeat(times) - a decorator factory - that runs the function the given number of times.

Solution:

```
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for i in range(times):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def remind(task):
    print("Reminder:", task)

remind("Submit assignment")
```

Output:

```
Reminder: Submit assignment
Reminder: Submit assignment
Reminder: Submit assignment
```

Q42. Retry Decorator

ADVANCED

Write retry(attempts) that re-runs a function when it raises an exception, up to the given attempts. Simulate a network call that fails twice then succeeds.

Solution:

```
def retry(attempts):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for n in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except ConnectionError as e:
                    print("Attempt", n, "failed:", e)
            return "All attempts failed"
        return wrapper
    return decorator

responses = [False, False, True]

@retry(3)
def fetch_data():
    if not responses.pop(0):
        raise ConnectionError("Network down")
    return "Data received"

print(fetch_data())
```

Output:

```
Attempt 1 failed: Network down
Attempt 2 failed: Network down
Data received
```

8. Multiple & Class Decorators

Q43. Two Decorators on One Function

BEGINNER

Apply bold and italic decorators (which wrap text in and) to a function returning "Bikkad IT".

Solution:

```
def bold(func):
    def wrapper():
        return "<b>" + func() + "</b>"
    return wrapper

def italic(func):
    def wrapper():
        return "<i>" + func() + "</i>"
    return wrapper

@bold
@italic
def brand():
    return "Bikkad IT"

print(brand())
```

Output:

```
<b><i>Bikkad IT</i></b>
```

Q44. Order Matters

BEGINNER

Swap the order of @bold and @italic and print the result to see how the order changes the output.

Solution:

```
def bold(func):
    def wrapper():
        return "<b>" + func() + "</b>"
    return wrapper

def italic(func):
    def wrapper():
        return "<i>" + func() + "</i>"
    return wrapper

@italic
@bold
def brand():
    return "Bikkad IT"

print(brand())
```

Output:

```
<i><b>Bikkad IT</b></i>
```

Q45. Execution Order of Stacked Decorators

INTERMEDIATE

Write decorators first and second that print "first before"/"second before" etc. Stack them and observe the printing order.

Solution:

```
def first(func):
    def wrapper():
        print("first before")
        func()
        print("first after")
    return wrapper

def second(func):
    def wrapper():
        print("second before")
        func()
        print("second after")
    return wrapper

@first
@second
def task():
    print("task running")

task()
```

Output:

```
first before
second before
task running
second after
first after
```

Q46. Class as a Decorator

INTERMEDIATE

Write class CallLogger with __init__(func) and __call__ that prints "Calling" and the function name before running it.

Solution:

```
class CallLogger:
    def __init__(self, func):
        self.func = func

    def __call__(self, *args, **kwargs):
        print("Calling", self.func.__name__)
        return self.func(*args, **kwargs)

@CallLogger
def multiply(a, b):
    return a * b

print(multiply(6, 7))
```

Output:

```
Calling multiply
42
```

Q47. Class Decorator with State

ADVANCED

Write class RateLimit(func) that allows a function to run at most 2 times; after that it prints "Limit reached".

Solution:

```
class RateLimit:
    def __init__(self, func):
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        if self.count >= 2:
            print("Limit reached")
            return None
        self.count += 1
        return self.func(*args, **kwargs)

@RateLimit
def send_otp(number):
    print("OTP sent to", number)

send_otp("98111")
send_otp("98111")
send_otp("98111")
```

Output:

```
OTP sent to 98111
OTP sent to 98111
Limit reached
```

Q48. Caching Decorator

ADVANCED

Write a memoize decorator that stores results in a dictionary so a slow Fibonacci function becomes fast. Also compare with functools.lru_cache.

Solution:

```
from functools import lru_cache

def memoize(func):
    cache = {}
    def wrapper(n):
        if n not in cache:
            cache[n] = func(n)
        return cache[n]
    return wrapper

@memoize
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

@lru_cache(maxsize=None)
def fib2(n):
    if n < 2:
        return n
    return fib2(n - 1) + fib2(n - 2)

print(fib(40))
print(fib2(40))
```

Output:

```
102334155
102334155
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026